

# Chapter 53 — Game Theory

## 0. What You Will Learn in This Chapter

Game Theory problems ask you to reason about optimal play between players.

They often involve:

- two players
- alternating turns
- perfect information
- both players play optimally
- winning and losing states
- DP over states
- minimax reasoning
- parity tricks
- Nim XOR
- Stone Game variants

In interviews, game theory can appear as:

- Can first player win?
- What is the optimal score difference?
- Is a position winning or losing?
- Which move forces victory?
- What happens under optimal play?

DE Shaw-style interviews may include reasoning-heavy game problems, especially involving invariants, parity, and expected strategy.

By the end of this chapter, you should understand:

- Winning and losing states
- Optimal play
- Minimax logic
- DP for games
- Score-difference DP
- Nim game
- XOR theorem
- Take-away games
- Stone Game patterns
- Strategy stealing idea
- Sprague-Grundy preview
- How to recognize game theory problems

Core problems covered in this chapter:

1. Nim Game
2. Divisor Game
3. Predict the Winner
4. Stone Game
5. Stone Game II
6. Stone Game III
7. Stone Game IV
8. Can I Win
9. Cat and Mouse, preview
10. Flip Game II
11. Winner Square Game
12. Take Away Stones

- 13. Sprague-Grundy preview
  - 14. Minimax preview
  - 15. Strategy stealing preview
- 

# 1. What Is Game Theory?

Game theory studies decision-making between players.

In algorithm interviews, most game theory problems have:

two players  
turns alternate  
both play optimally  
full information  
no randomness

Example:

There are 10 stones.

Each turn, a player removes 1 to 3 stones.

Player who removes the last stone wins.

Question:

Can the first player force a win?

This is a game state problem.

---

# 2. Winning and Losing States

The most important game theory idea:

A state is winning if the current player can move to at least one losing state.

A state is losing if every legal move goes to a winning state.

Formally:

$\text{winning}[\text{state}] = \text{exists move such that } \text{winning}[\text{nextState}] == \text{false}$

Base case:

state with no legal moves is losing

because current player cannot move.

---

# 3. Example — Remove 1 to 3 Stones

Let:

$\text{dp}[n]$  = whether current player wins with  $n$  stones

If  $n = 0$ :

$\text{dp}[0] = \text{false}$

because no move.

For  $n$ :

```
dp[n] = !dp[n - 1] || !dp[n - 2] || !dp[n - 3]
```

if those states exist.

Values:

n=0 losing  
n=1 winning  
n=2 winning  
n=3 winning  
n=4 losing  
n=5 winning  
n=6 winning  
n=7 winning  
n=8 losing

Pattern:

multiples of 4 are losing

## 4. Optimal Play

"Both players play optimally" means:

- Current player chooses the move best for themselves.
- Opponent will also choose their best response.
- You cannot assume opponent makes mistakes.

This often leads to:

minimax

DP

winning/losing state recurrence

## 5. Common Game DP Forms

|                                  |                  |
|----------------------------------|------------------|
|                                  |                  |
| Can current player win?          | bool dp[state]   |
| Max score current player can get | int dp[state]    |
| Score difference                 | int dp[l][r]     |
| Used numbers                     | bitmask dp[mask] |
| Stone piles                      | interval DP      |
| Take-away game                   | dp[stones]       |
| Nim-like game                    | XOR              |

|            |                          |
|------------|--------------------------|
| Game Type  | Common State             |
| Graph game | state = positions + turn |

## 6. Pattern 1 — Boolean Winning State DP

Use when problem asks:

Can first player win?

Recurrence:

`dp[state] = false;`

for next in moves:

    if `dp[next] == false:`

`dp[state] = true;`

Meaning:

If I can move opponent to a losing state, I win.

## 7. Pattern 2 — Score Difference DP

Use when both players collect scores.

Instead of tracking both scores, track:

current player's score - opponent's score

For interval `[l, r]`:

If take left:

`nums[l] - dp[l + 1][r]`

Why subtract?

After current player takes `nums[l]`, the opponent becomes current player on remaining interval. `dp[l + 1][r]` is opponent's advantage from that state.

So current advantage is:

`nums[l] - opponentAdvantage`

Recurrence:

`dp[l][r] = max(nums[l] - dp[l + 1][r],  
                  nums[r] - dp[l][r - 1])`

First player wins if:

`dp[0][n - 1] >= 0`

## 8. Pattern 3 — Minimax

Minimax is used when:

- one player maximizes outcome
- other player minimizes outcome

Generic:

best = max over moves of opponent result

or:

best = min over opponent moves

In many interview games, score-difference DP is simpler than explicit minimax.

## 9. Pattern 4 — Bitmask Game DP

Use when state is a set of chosen/remaining numbers.

Example:

Can I Win

State:

mask = which numbers have been used

Memoized DFS:

canWin(mask, remaining)

Current player wins if they can choose a number that immediately wins or leaves opponent losing.

## 10. Pattern 5 — Nim XOR

Nim game:

- There are piles of stones.
- On each turn, choose one pile and remove positive stones.
- Player unable to move loses.

The theorem:

First player wins iff xor of all pile sizes  $\neq 0$

This is one of the most famous game theory results.

## 11. Pattern Recognition Checklist

Ask:

16. Are there two players?
17. Do turns alternate?
18. Does both play optimally appear?
19. Is it asking if first player can win?
20. Is it asking maximum score or score difference?
21. Is state small enough for DP?
22. Is there a pile/take-away structure?
23. Is it Nim-like with XOR?
24. Are choices from left/right ends?

- 25. Are used numbers naturally represented by a bitmask?
  - 26. Is there a graph position and turn?
- 

## 12. Full Problem Walkthrough 1 — LeetCode 292: Nim Game

### 12.1 Problem Summary

There are  $n$  stones.  
Each turn remove 1, 2, or 3 stones.  
Player who removes last stone wins.  
Can first player win?

### 12.2 Key Insight

Multiples of 4 are losing.  
If  $n = 4$ :  
No matter whether first removes 1, 2, or 3, opponent removes the rest.  
If  $n$  is not multiple of 4:  
First player removes:

$n \% 4$

stones, leaving a multiple of 4.

### 12.3 C++ Code

```
class Solution {  
public:  
    bool canWinNim(int n) {  
        return n % 4 != 0;  
    }  
};
```

### 12.4 Complexity

Time:

$O(1)$

Space:

$O(1)$

## 13. Full Problem Walkthrough 2 — LeetCode 1025: Divisor Game

### 13.1 Problem Summary

Alice starts with  $n$ .  
On each turn, choose  $x$  such that:

```
0 < x < n
n % x == 0
```

Then replace n by:

```
n - x
```

Player unable to move loses.

## 13.2 Math Insight

Alice wins iff n is even.

Why?

- If n is even, Alice can choose  $x = 1$ , leaving odd.
- From odd n, every divisor x is odd, so  $n - x$  is even.
- Alice can always give Bob odd, Bob gives back even.
- Eventually Bob reaches 1 and loses.

## 13.3 C++ Code

```
class Solution {
public:
    bool divisorGame(int n) {
        return n % 2 == 0;
    }
};
```

## 13.4 DP Alternative

```
class Solution {
public:
    bool divisorGame(int n) {
        vector<bool> dp(n + 1, false);

        for (int i = 2; i <= n; i++) {
            for (int x = 1; x < i; x++) {
                if (i % x == 0 && dp[i - x] == false) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[n];
    }
};
```

# 14. Full Problem Walkthrough 3 — LeetCode 486: Predict the Winner

## 14.1 Problem Summary

Players take numbers from either end of array.

Each wants maximum total score.

Return whether first player can win or tie.

## 14.2 Score Difference DP

State:

$dp[l][r]$  = max score difference current player can achieve from  $nums[l..r]$

Transition:

$takeLeft = nums[l] - dp[l + 1][r]$

$takeRight = nums[r] - dp[l][r - 1]$

$dp[l][r] = \max(takeLeft, takeRight)$

## 14.3 C++ Code

```
class Solution {
public:
    bool predictTheWinner(vector<int>& nums) {
        int n = nums.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = nums[i];
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                int takeLeft = nums[l] - dp[l + 1][r];
                int takeRight = nums[r] - dp[l][r - 1];

                dp[l][r] = max(takeLeft, takeRight);
            }
        }

        return dp[0][n - 1] >= 0;
    }
};
```

## 14.4 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

# 15. Full Problem Walkthrough 4 — LeetCode 877: Stone Game

## 15.1 Problem Summary

There are even number of piles.

Players take from either end.

Total stones is odd.

Return whether Alice wins.

## 15.2 Strategy Stealing Insight

Alice always wins.

Why?

Because there are even number of piles.

Alice can choose to take all piles originally at even indices or all piles originally at odd indices, depending on which sum is larger.

Since total is odd, one parity has larger sum.

Alice can force that parity.

## 15.3 C++ Code

```
class Solution {  
public:  
    bool stoneGame(vector<int>& piles) {  
        return true;  
    }  
};
```

## 15.4 Interview Note

You can also solve with score-difference DP, but the math insight is expected.

# 16. Full Problem Walkthrough 5 — LeetCode 1140: Stone Game II

## 16.1 Problem Summary

There are piles.

Alice and Bob take turns.

At each turn, player can take first  $X$  remaining piles where:

$1 \leq X \leq 2M$

Then:

$M = \max(M, X)$

Return maximum stones Alice can get.

## 16.2 State

$dp[i][M]$  = maximum stones current player can get from piles[i..]

Let  $suffix[i]$  be total stones from i to end.

If current player takes X piles, opponent gets:

$dp[i + X][\max(M, X)]$

So current gets:

$suffix[i] - \text{opponentBest}$

Transition:

$dp[i][M] = \max \text{ over } X \text{ of } suffix[i] - dp[i + X][\max(M, X)]$

## 16.3 C++ Code

```
class Solution {
public:
    int stoneGameII(vector<int>& piles) {
        int n = piles.size();

        vector<int> suffix(n + 1, 0);

        for (int i = n - 1; i >= 0; i--) {
            suffix[i] = suffix[i + 1] + piles[i];
        }

        vector<vector<int>> memo(n, vector<int>(n + 1, -1));

        function<int(int, int)> dfs = [&](int i, int M) {
            if (i >= n) {
                return 0;
            }

            if (2 * M >= n - i) {
                return suffix[i];
            }

            if (memo[i][M] != -1) {
                return memo[i][M];
            }
        };
    }
};
```

```

    int best = 0;

    for (int X = 1; X <= 2 * M; X++) {
        int opponent = dfs(i + X, max(M, X));
        best = max(best, suffix[i] - opponent);
    }

    memo[i][M] = best;

    return best;
};

return dfs(0, 1);
}
};

```

## 16.4 Complexity

States:

$O(n^2)$

Transitions up to:

$O(n)$

Worst-case:

$O(n^3)$

Often accepted for constraints.

# 17. Full Problem Walkthrough 6 — LeetCode 1406: Stone Game III

## 17.1 Problem Summary

Players take 1, 2, or 3 stones from the front.

Each stone has value, can be negative.

Return:

"Alice", "Bob", or "Tie"

depending on optimal play.

## 17.2 Score Difference DP

State:

$dp[i]$  = max score difference current player can achieve starting at  $i$

Try taking 1 to 3 stones:

$takeSum - dp[i + k]$

## 17.3 C++ Code

```
class Solution {
public:
    string stoneGameIII(vector<int>& stoneValue) {
        int n = stoneValue.size();

        vector<int> dp(n + 1, 0);

        for (int i = n - 1; i >= 0; i--) {
            int take = 0;
            dp[i] = INT_MIN;

            for (int k = 1; k <= 3 && i + k <= n; k++) {
                take += stoneValue[i + k - 1];

                dp[i] = max(dp[i], take - dp[i + k]);
            }
        }

        if (dp[0] > 0) {
            return "Alice";
        }

        if (dp[0] < 0) {
            return "Bob";
        }

        return "Tie";
    }
};
```

## 17.4 Lesson

When scores can be negative, score-difference DP still works well.

---

# 18. Full Problem Walkthrough 7 — LeetCode 1510: Stone Game IV / Winner Square Game

## 18.1 Problem Summary

There are  $n$  stones.

Each turn, remove a non-zero square number of stones.

Player unable to move loses.  
Return whether first player wins.

## 18.2 Winning State DP

$dp[i]$  = true if current player can win with  $i$  stones

Transition:

$dp[i]$  = exists square  $s \leq i$  such that  $dp[i - s] == \text{false}$

## 18.3 C++ Code

```
class Solution {
public:
    bool winnerSquareGame(int n) {
        vector<bool> dp(n + 1, false);

        for (int stones = 1; stones <= n; stones++) {
            for (int x = 1; x * x <= stones; x++) {
                int square = x * x;

                if (dp[stones - square] == false) {
                    dp[stones] = true;
                    break;
                }
            }
        }

        return dp[n];
    }
};
```

## 18.4 Complexity

Time:

$O(n \sqrt{n})$

Space:

$O(n)$

# 19. Full Problem Walkthrough 8 — LeetCode 464: Can I Win

## 19.1 Problem Summary

Players choose numbers from 1 to `maxChoosableInteger`.  
Each number can be chosen once.

Player who causes running total to reach or exceed `desiredTotal` wins.

Return whether first player can force win.

## 19.2 State

Used numbers are represented by bitmask.

mask = chosen numbers

remaining = amount needed to win

Current player wins if they can pick a number:

- = remaining immediately, or
- leaves opponent in losing state.

## 19.3 Early Checks

If total sum of all numbers is less than desiredTotal:

false

If desiredTotal <= 0:

true

## 19.4 C++ Code

```
class Solution {
public:
    unordered_map<int, bool> memo;
    int maxNum;

    bool canIWin(int maxChoosableInteger, int desiredTotal) {
        maxNum = maxChoosableInteger;

        if (desiredTotal <= 0) {
            return true;
        }

        int total = maxNum * (maxNum + 1) / 2;

        if (total < desiredTotal) {
            return false;
        }

        return dfs(0, desiredTotal);
    }

    bool dfs(int mask, int remaining) {
        if (memo.count(mask)) {
            return memo[mask];
        }
    }
```

```

for (int num = 1; num <= maxNum; num++) {
    int bit = 1 << (num - 1);

    if ((mask & bit) == 0) {
        if (num >= remaining) {
            memo[mask] = true;
            return true;
        }

        if (!dfs(mask | bit, remaining - num)) {
            memo[mask] = true;
            return true;
        }
    }

    memo[mask] = false;
    return false;
}
};

```

### 19.5 Why Memo Key Only Mask?

Given original desiredTotal and chosen numbers, remaining is determined by mask. So mask alone is enough.

## 20. Full Problem Walkthrough 9 — Flip Game II

### 20.1 Problem Summary

Given string with + and -.

Players alternate flipping any "++" into "--".

Player unable to move loses.

Return whether first player can guarantee win.

### 20.2 State

String itself is state.

Use memoized DFS.

### 20.3 C++ Code

```

class Solution {
public:
    unordered_map<string, bool> memo;

    bool canWin(string currentState) {

```

```

    if (memo.count(currentState)) {
        return memo[currentState];
    }

    for (int i = 0; i + 1 < currentState.size(); i++) {
        if (currentState[i] == '+' && currentState[i + 1] == '+') {
            string next = currentState;

            next[i] = '-';
            next[i + 1] = '-';

            if (!canWin(next)) {
                memo[currentState] = true;
                return true;
            }
        }
    }

    memo[currentState] = false;
    return false;
}
};

```

## 20.4 Lesson

If state space is not numeric but small enough, memoize the string.

# 21. Nim Game Deep Dive

## 21.1 Problem

Multiple piles.

Each move:

- choose one pile
- remove at least one stone

Player unable to move loses.

## 21.2 XOR Theorem

Let:

$\text{xorSum} = \text{pile1} \oplus \text{pile2} \oplus \dots \oplus \text{pileN}$

If:

$\text{xorSum} == 0$

current player is losing.

If:

$\text{xorSum} \neq 0$

current player is winning.

### 21.3 Why $\text{xorSum} == 0$ Is Losing

Any move changes one pile.

Changing one pile from  $x$  to  $y$  changes  $\text{xor}$  from:

$\text{xorSum}$

to:

$\text{xorSum} \oplus x \oplus y$

If  $\text{xorSum}$  was 0, new  $\text{xor}$  becomes:

$x \oplus y$

Since  $x \neq y$ , this is non-zero.

So every move from zero  $\text{xor}$  goes to non-zero  $\text{xor}$ .

### 21.4 Why $\text{xorSum} \neq 0$ Is Winning

There is a move that makes  $\text{xor}$  zero.

Let highest set bit of  $\text{xorSum}$  be  $b$ .

There must be a pile with bit  $b$  set.

For that pile  $x$ , set:

$y = x \oplus \text{xorSum}$

Then:

$y < x$

so we can reduce pile  $x$  to  $y$ .

New  $\text{xor}$  becomes zero.

## 22. Full Problem Walkthrough 10 — Take Away Stones Nim Variant

If each move can remove 1 to  $k$  stones from one pile, losing states for one pile are:

$n \% (k + 1) == 0$

For multiple independent piles, Sprague-Grundy theory generalizes this.

For simple Nim, pile Grundy value is pile size.

For remove 1.. $k$ , pile Grundy value is:

$n \% (k + 1)$

Then XOR Grundy values.

This is advanced but useful.

## 23. Sprague-Grundy Theorem Preview

### 23.1 What It Says

Every impartial game position is equivalent to a Nim pile of some size called its Grundy number.

For a state:

grundy[state] = mex({grundy[next] for all moves})

mex means minimum excluded non-negative integer.

Example:

If reachable Grundy values are:

{0, 1, 3}

mex is:

2

## 23.2 Winning Rule

For a combination of independent games:

xor of Grundy numbers != 0

means winning.

## 23.3 Interview Note

Sprague-Grundy is advanced.

Most interviews do not require full implementation, but recognizing Nim-like XOR is valuable.

---

# 24. Cat and Mouse Preview

Graph game problems involve state:

mouse position

cat position

turn

These are harder because cycles can cause draws.

Often solved by:

- BFS from terminal states
- retrograde analysis
- DP with move limits

This is advanced and should be treated separately.

---

# 25. Strategy Stealing Preview

Strategy stealing is a proof technique.

It shows first player can win by assuming second player has winning strategy, then first makes an arbitrary move and adopts that strategy.

Used in some board games and mathematical proofs.

In coding interviews, it appears less often than DP or Nim.

---

## 26. Common Game Theory Edge Cases

### 26.1 No Moves

Usually losing state.

### 26.2 Tie Allowed

For score games, first player may win if:

`scoreDifference >= 0`

not only  $> 0$ .

### 26.3 Negative Scores

Score-difference DP handles negative values better than greedy.

### 26.4 Total Sum Too Small

In Can I Win, if all numbers cannot reach target, first cannot win.

### 26.5 Memoization State

Include all variables that affect future game.

If a variable is determined by mask, it may not need separate memo key.

---

## 27. Common Game Theory Mistakes

### Mistake 1 — Greedy Move

Choosing largest immediate score may lose.

Optimal play requires considering opponent response.

### Mistake 2 — Forgetting Opponent Also Optimal

Do not assume opponent makes weak moves.

### Mistake 3 — Wrong Base Case

No legal moves usually means losing.

### Mistake 4 — Score DP Instead of Difference DP

Tracking only current score can be misleading.

Use score difference.

### Mistake 5 — Missing Memoization

Game recursion without memo often explodes exponentially.

### Mistake 6 — Applying Nim XOR Incorrectly

Nim XOR applies to independent impartial games under normal play.

Not every stone game is Nim.

---

## 28. Interview Explanation Templates

### 28.1 Winning State DP

A state is winning if there exists at least one move that puts the opponent into a losing state. If all moves give the opponent winning states, the current state is losing.

## 28.2 Score Difference DP

I store the maximum score difference current player can achieve. When current player takes a value, the remaining DP represents opponent's advantage, so I subtract it.

## 28.3 Bitmask Game

The chosen numbers form a bitmask. I recursively try every unused number. If any move wins immediately or leaves the opponent unable to win, current state is winning.

## 28.4 Nim

The key invariant is xor of pile sizes. Zero xor is losing because every move makes it nonzero. Nonzero xor is winning because there is always a move to make it zero.

# 29. Complexity Patterns

| Nim Game simple      | $O(1)$                | $O(1)$    |
|----------------------|-----------------------|-----------|
| Boolean take-away DP | $O(n * \text{moves})$ | $O(n)$    |
| Score interval DP    | $O(n^2)$              | $O(n^2)$  |
| Stone Game II        | $O(n^3)$ typical      | $O(n^2)$  |
| Stone Game III       | $O(n)$                | $O(n)$    |
| Winner Square Game   | $O(n \sqrt{n})$       | $O(n)$    |
| Can I Win            | $O(2^n * n)$          | $O(2^n)$  |
| Flip Game II         | exponential states    | memo size |

# 30. Expanded Practice Set

## 30.1 Must-Do Easy / Medium

27. Nim Game
28. Divisor Game
29. Predict the Winner
30. Stone Game
31. Stone Game III
32. Stone Game IV
33. Flip Game II

## 30.2 Strong Interview Practice

34. Can I Win
35. Stone Game II
36. Stone Game VII

- 37. Stone Game VIII
- 38. Guess Number Higher or Lower II
- 39. New 21 Game, probability/game DP hybrid

### 30.3 Hard / Stretch

- 40. Cat and Mouse
  - 41. Cat and Mouse II
  - 42. Sprague-Grundy examples
  - 43. Composite impartial games
  - 44. Strategy stealing proof problems
- 

## 31. Suggested Study Order

### Step 1 — Winning/Losing Basics

Solve:

- 45. Nim Game
- 46. Divisor Game
- 47. Winner Square Game

Goal:

Understand winning states.

### Step 2 — Score Difference DP

Solve:

- 48. Predict the Winner
- 49. Stone Game III
- 50. Stone Game VII

Goal:

Think in score advantage.

### Step 3 — Bitmask Games

Solve:

- 51. Can I Win
- 52. Flip Game II

Goal:

Represent game state compactly.

### Step 4 — Advanced Stone Games

Solve:

- 53. Stone Game II
- 54. Stone Game VIII

Goal:

Handle complex game DP.

### Step 5 — Nim / Grundy

Study:

- 55. Nim theorem
- 56. Sprague-Grundy preview
- 57. take-away game variants

Goal:

Recognize XOR-based game states.

---

## 32. Pattern Decision Table

|                          |                              |
|--------------------------|------------------------------|
|                          |                              |
| remove 1 to k stones     | modulo $k+1$ / DP            |
| no moves loses           | winning-state DP             |
| pick from ends for score | interval score-difference DP |
| take first 1..3 values   | 1D score-difference DP       |
| choose unused numbers    | bitmask memo DFS             |
| independent piles        | Nim XOR / Grundy             |
| square removals          | boolean DP                   |
| graph positions + turns  | graph game DP                |
| even piles, odd total    | parity / strategy stealing   |
| return Alice/Bob/Tie     | score difference sign        |

## 33. Mini Problem 1 — Remove 1 or 2 Stones

Player removes 1 or 2 stones.

Losing states:

```
n % 3 == 0
```

Code:

```
bool firstWins(int n) {  
    return n % 3 != 0;  
}
```

## 34. Mini Problem 2 — Remove 1 to k Stones

```
bool firstWins(int n, int k) {  
    return n % (k + 1) != 0;  
}
```

The winning move is to leave a multiple of  $k + 1$ .

## 35. Mini Problem 3 — Score Difference From Ends

```
int maxDifference(vector<int>& nums) {
    int n = nums.size();

    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        dp[i][i] = nums[i];
    }

    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;

            dp[l][r] = max(nums[l] - dp[l + 1][r],
                           nums[r] - dp[l][r - 1]);
        }
    }

    return dp[0][n - 1];
}
```

## 36. Mini Problem 4 — Mex Function

Used in Sprague-Grundy.

```
int mex(vector<int>& values) {
    unordered_set<int> seen(values.begin(), values.end());

    int g = 0;

    while (seen.count(g)) {
        g++;
    }

    return g;
}
```

## 37. Common Edge Case Checklist

Before submitting game code, test:

n = 0  
n = 1  
only one move possible  
all moves winning for opponent  
at least one move losing for opponent  
tie condition  
negative values  
single pile  
multiple equal piles  
desiredTotal <= 0  
sum numbers < desiredTotal  
mask full

## 38. Debugging Game Theory

If game solution fails, check:

58. Is base case correct?
  59. Does current player win if any move makes opponent lose?
  60. Are you using all legal moves?
  61. Are you memoizing enough state?
  62. Is tie considered win or not?
  63. Are scores handled as difference?
  64. Are negative values possible?
  65. Is Nim theorem actually applicable?
  66. Are state transitions reducing the game?
- 

## 39. What to Master Before Chapter 54

Before moving to Design Data Structures, make sure you can:

- Explain winning and losing states
  - Solve Nim Game
  - Solve Divisor Game
  - Solve Predict the Winner
  - Use score-difference DP
  - Solve Stone Game III
  - Solve Winner Square Game
  - Solve Can I Win with bitmask DFS
  - Explain Nim XOR theorem
  - Recognize when not to use Nim
  - Understand Sprague-Grundy conceptually
-

# 40. Final Cheat Sheet

## Winning State

```
dp[state] = false;

for next in moves:
    if dp[next] == false:
        dp[state] = true;
```

## No Moves

losing state

## Remove 1 to k Stones

```
firstWins = n % (k + 1) != 0;
```

## Score Difference Interval

```
dp[l][r] = max(nums[l] - dp[l + 1][r],
               nums[r] - dp[l][r - 1]);
```

## Score Difference Linear

```
dp[i] = max(takeSum - dp[i + k]);
```

## Bitmask Can Win

```
if choose num wins immediately:
    return true

if opponent loses after choose num:
    return true
```

## Nim

```
xorSum = pile1 ^ pile2 ^ ... ^ pileN;

firstWins = xorSum != 0;
```

## Grundy

```
grundy[state] = mex(grundy[next states])
```

# 41. Core Takeaway

Game theory problems are about optimal response.  
The most important rule:

A state is winning if you can move to a losing state.  
A state is losing if every move goes to a winning state.

For score games, use score difference.

For used-number games, use bitmask memoization.

For independent pile games, look for Nim XOR.

Do not rely on greedy unless you can prove it.

Always ask:

What is the state?

What are legal moves?

What is the base losing state?

Can I memoize?

Is there a mathematical invariant?

Once you identify those pieces, most game problems become structured DP or a known theorem.

---